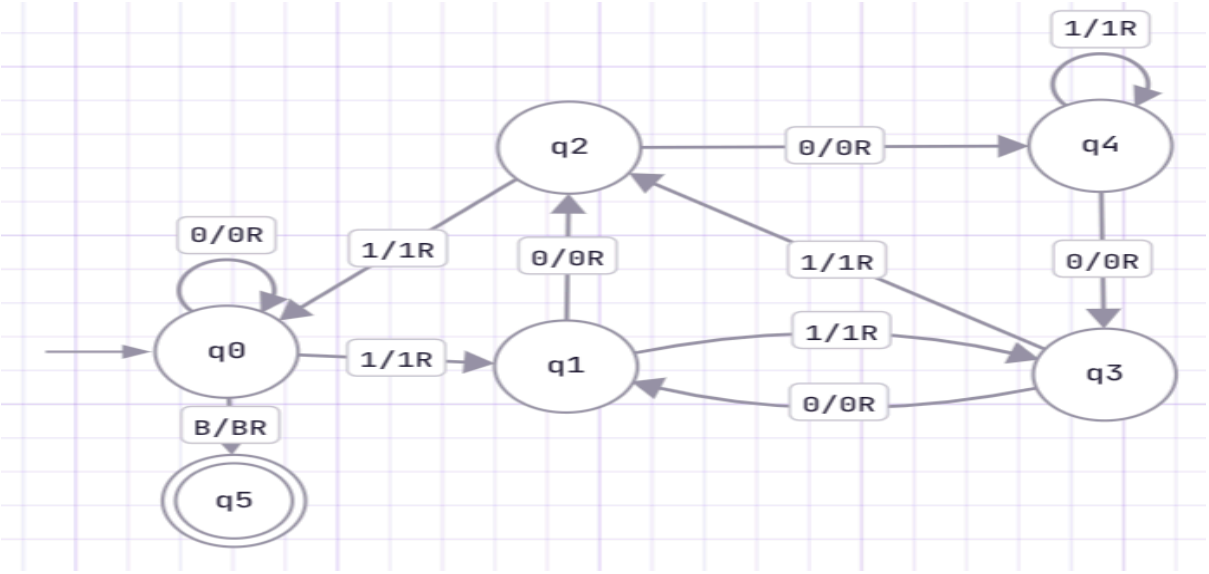
Personalization
ERP:35762 **R:** 62 **V:**2 **B:**111110 **n:**6

QUESTION NO:01

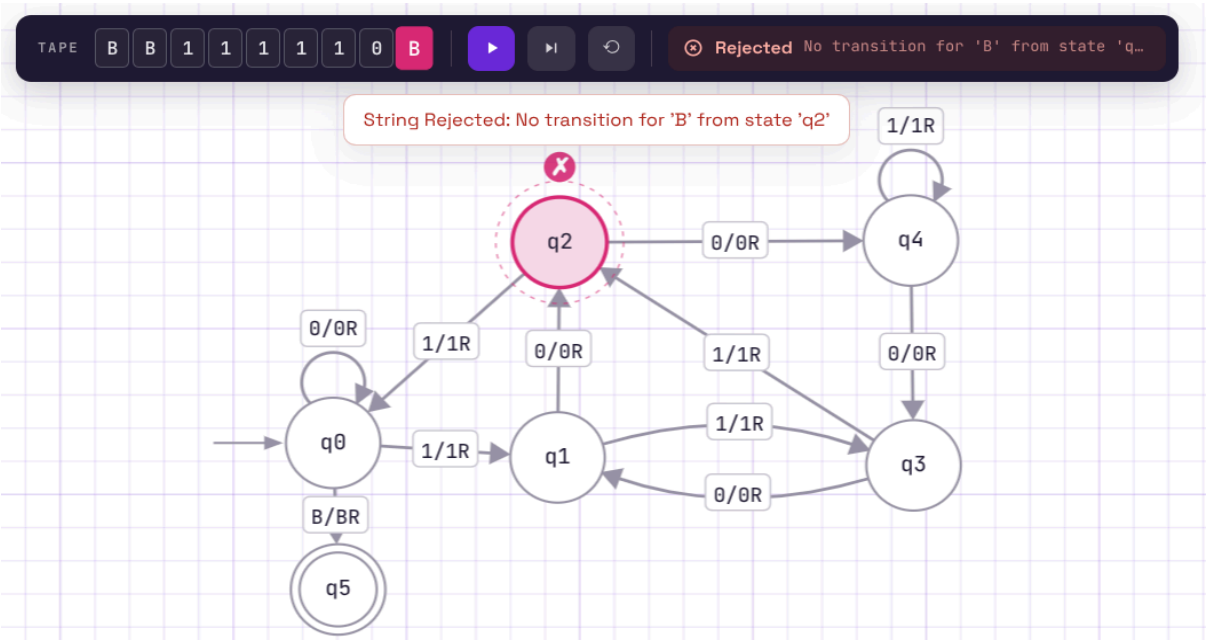
TASK:A

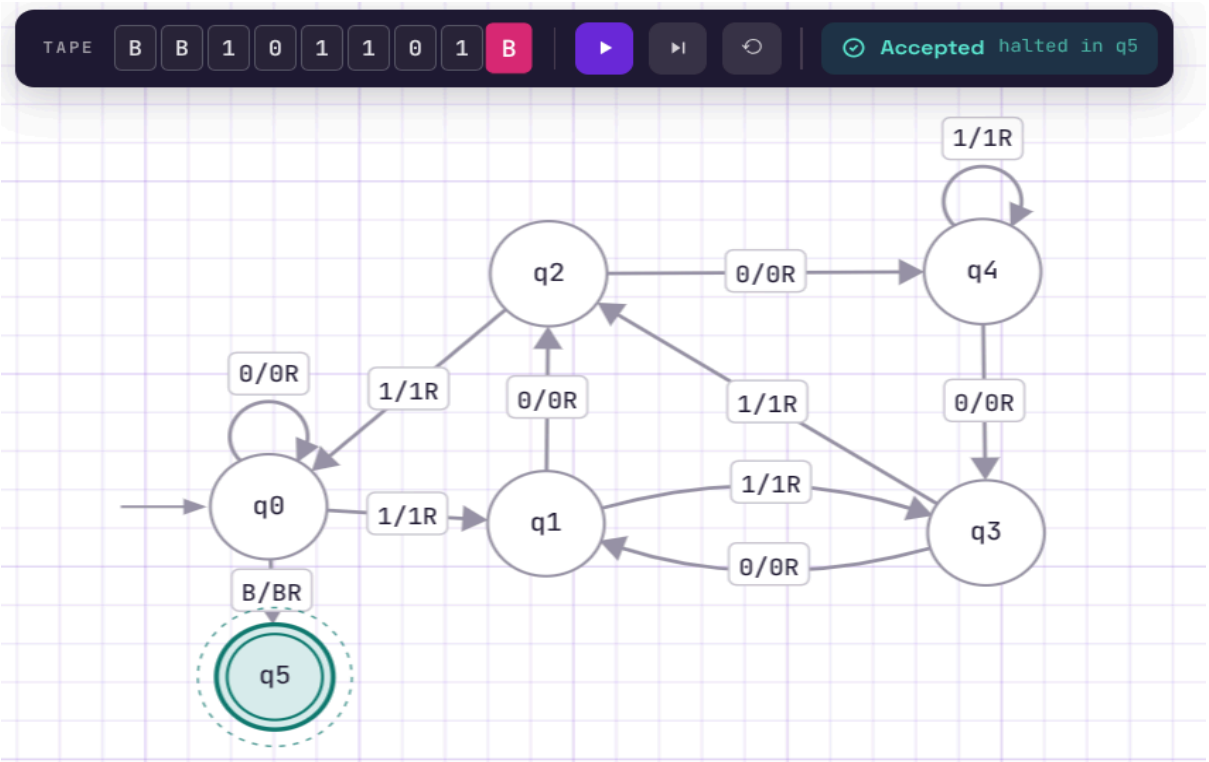
My TM machine keeps track of the remainder of the binary number while reading the input from left to right. It uses five working states, q0 through q4, where the state represents the current remainder 0 through 4. When a new binary digit is read, the new remainder is calculated using (2r + bit) mod 5. The machine moves right after reading every bit and stops when it reaches the blank symbol. It accepts if the final state is at q0 and rejects otherwise.

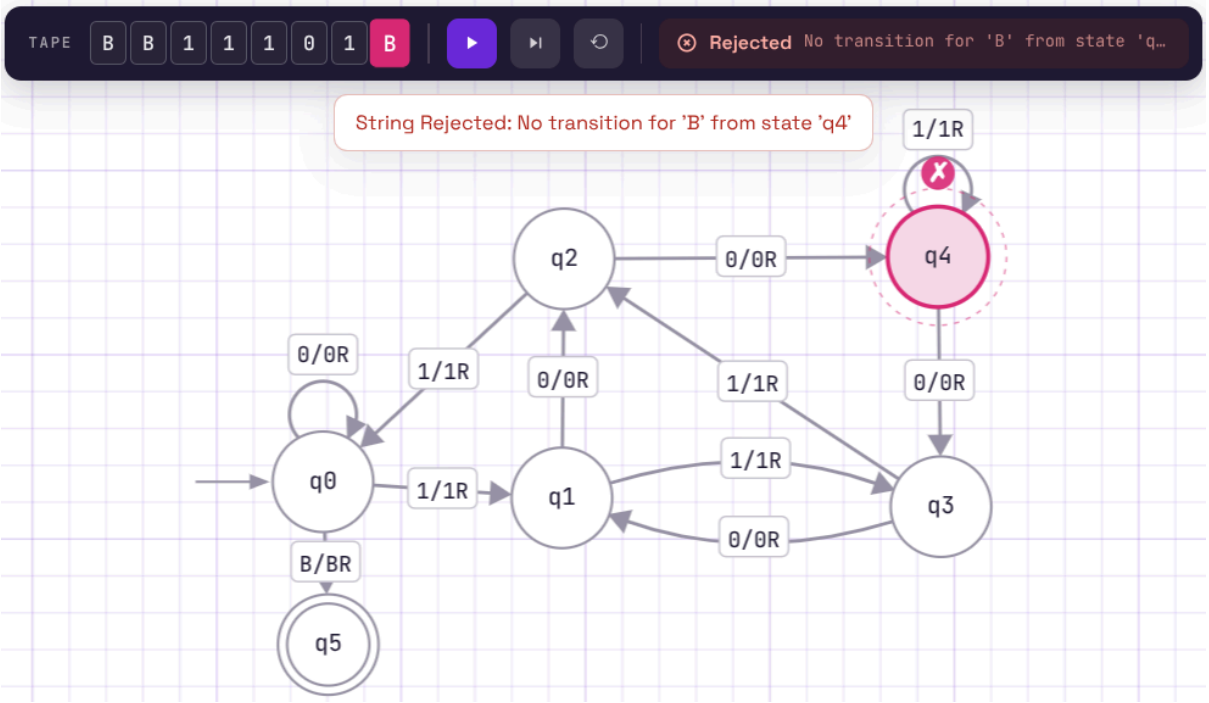
TASK:B



TASK:C







LINK:
https://www.automataverse.com/m/wx8mtbfc

TASK:D

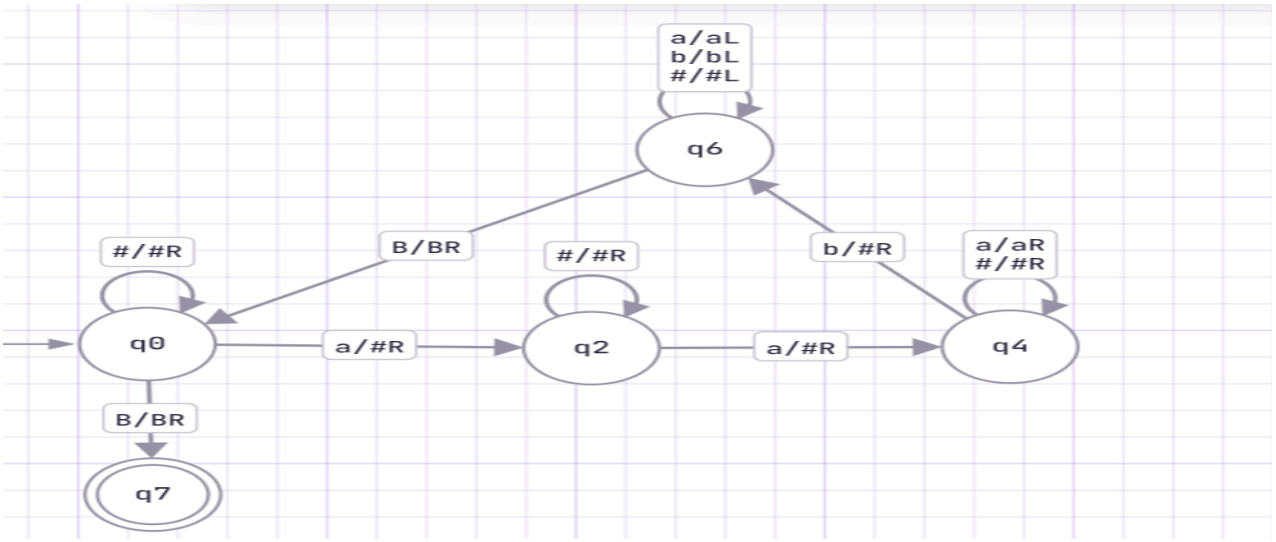
This TM machine designed for M=5 stores the remainder mod 5, so its states represent only the five possible values 0,1,2,3 and 4. If M changes, the machine needs a different set of remainder states and different transitions. Therefore, the TM machine cannot be reused and unmodified for another value of M.

QUESTION NO: 02

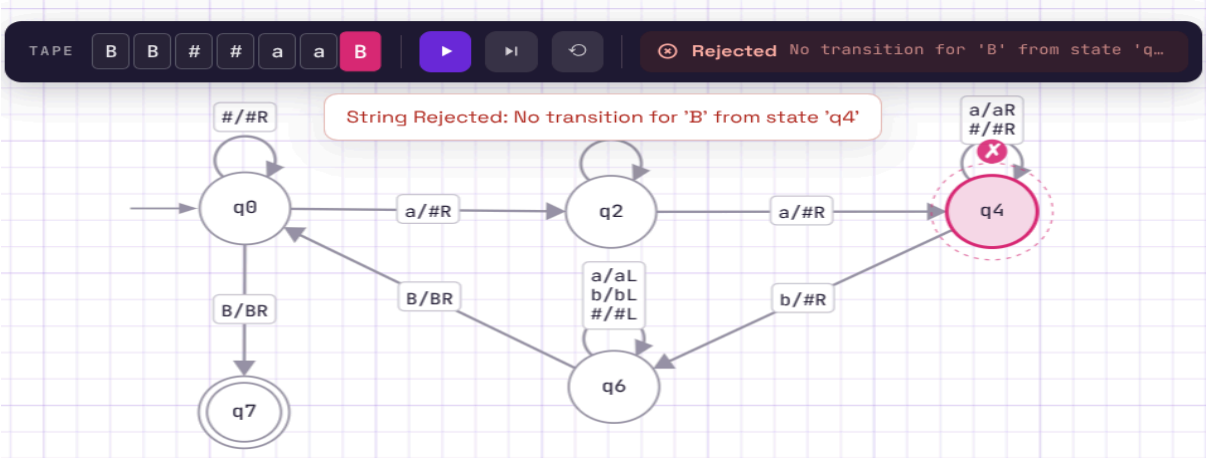
TASK:A

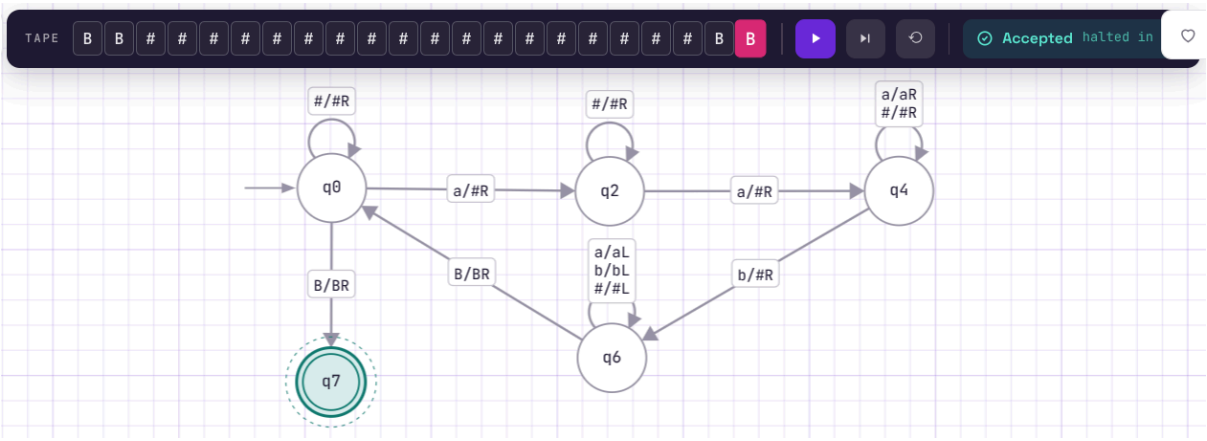
My TM Machine will find two A's and one B at a time. I use the patterns aab to identify these three symbols, and then I mark them as X. After I move back to the leftmost side of the tape and start the same process again. I keep doing this until there are no more symbols left to process. If all the A's and B's can be matched in groups of two A's and one B, the machine goes to the accept state. If I cannot find the required two A's and one B, or some symbols are left unmatched, the machine goes to the reject state.

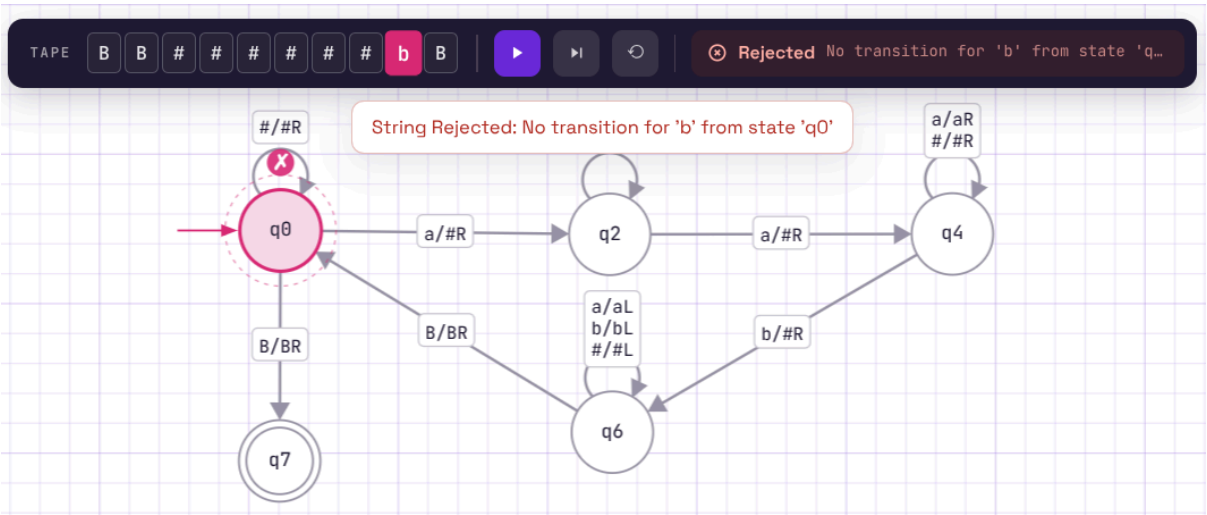
TASK:B



TASK:C







LINK:
https://www.automataverse.com/m/MGZUyxzM

TASK:D

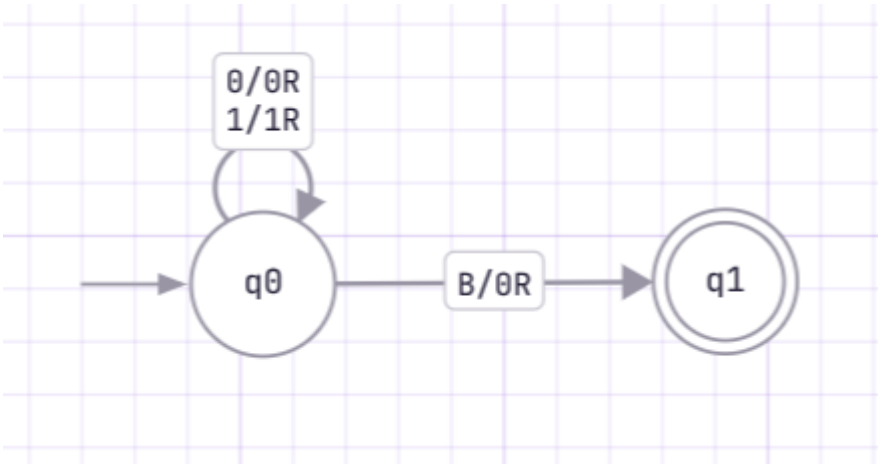
The off-by-one error would most likely happen when I choose the second A or the B in each group. I checked this by tracing different inputs step by step and making sure that every group marks exactly 2 A's and 1 B. I also checked that the machine returns to the left and starts the next group correctly.

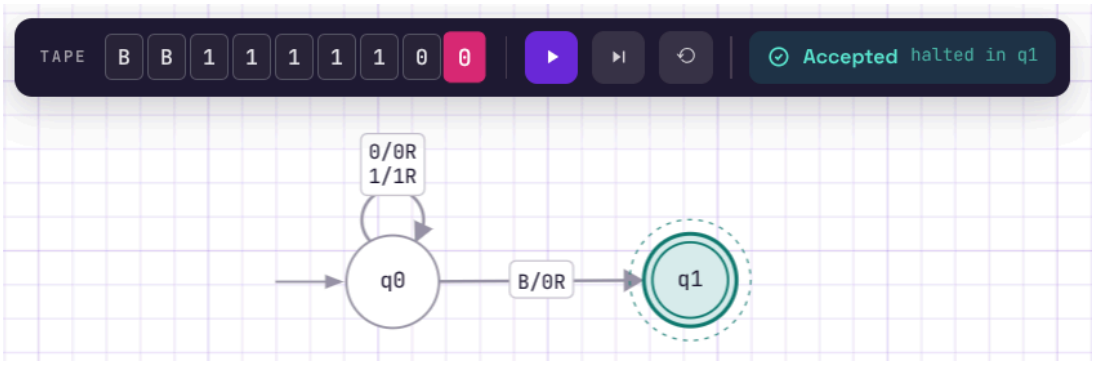
QUESTION NO: 03

TASK:A

Multiplying a binary number by 2 is equivalent to shifting all bits one position to the left and putting a `0` at the end. Because of this, I simply move the tape head from the beginning toward the right until it finds the blank after the binary number. At that blank, I write `0`. The original bits stay unchanged, so the final tape contains the original binary number followed by one zero.

TASK:B





LINK:
https://www.automataverse.com/m/R3bRdrmb

TASK:D

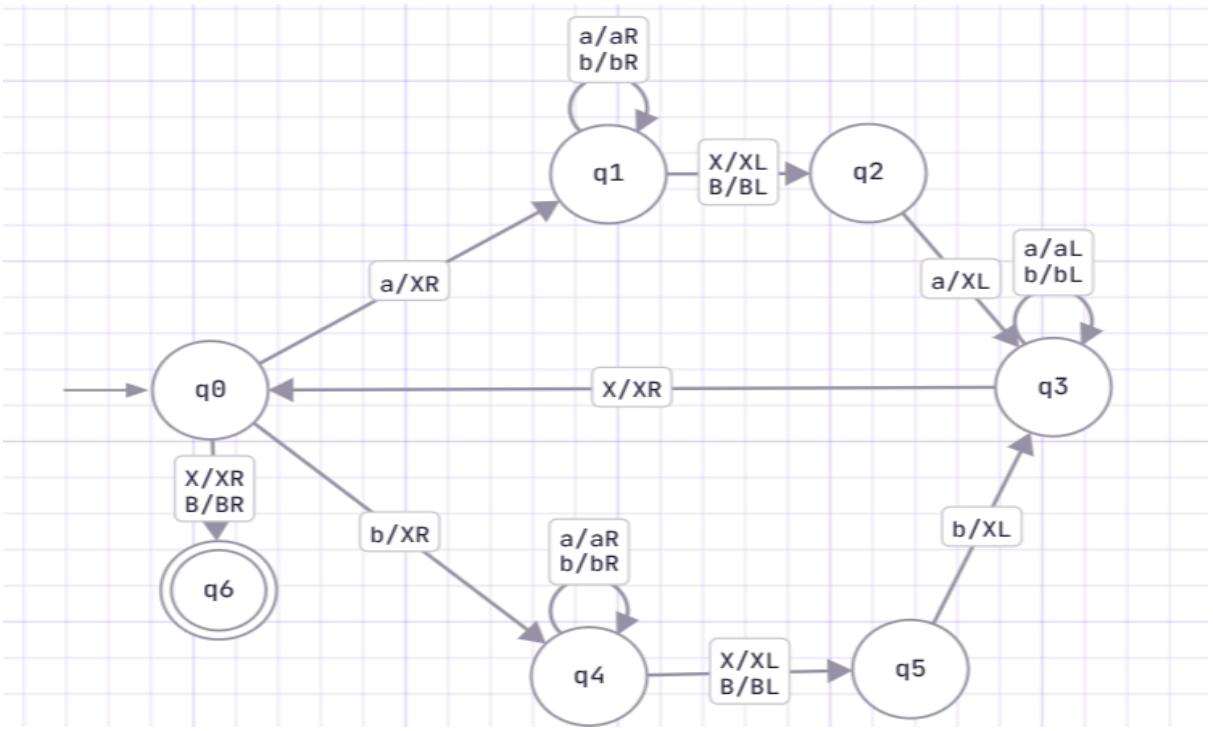
An important edge case is the binary number 1000. Multiplying it by 2 gives 10000, so changing only the leftmost bit would give the wrong result. The operation is a binary left shift, which means that every existing bit must stay unchanged and a 0 must be added at the right end.

QUESTION NO: 04

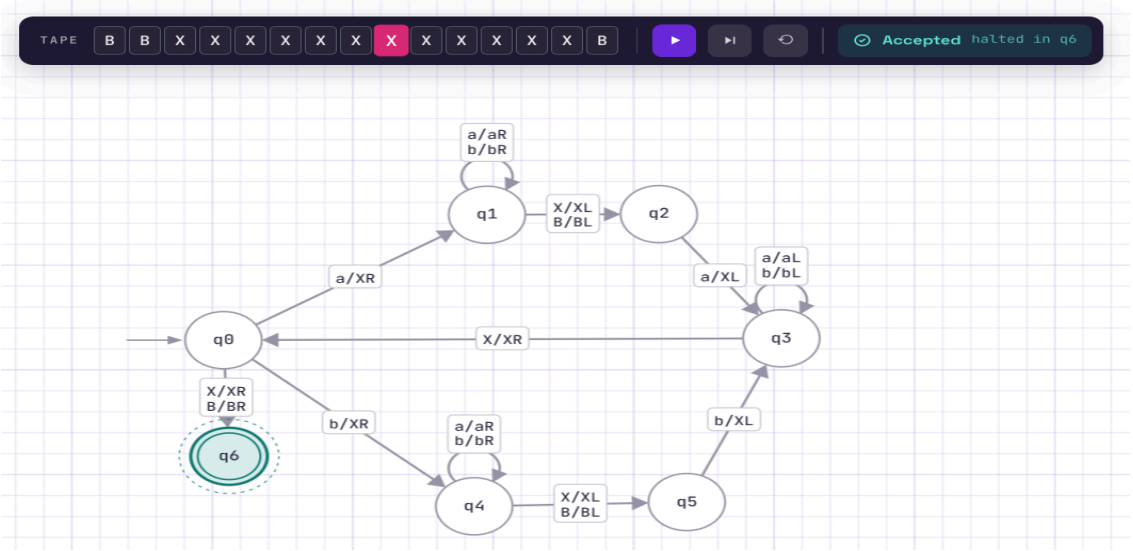
TASK:A

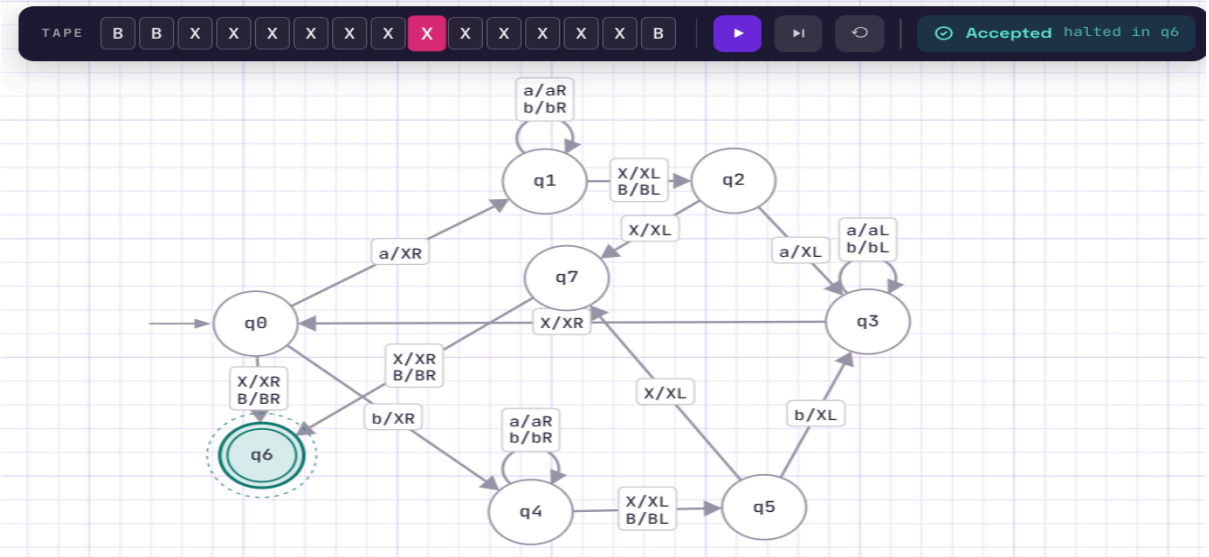
I use the standard palindrome strategy of comparing the outermost symbols and then moving inward. I mark the leftmost unused symbol and remember whether it was a or b. Then I move to the right end and find the rightmost unused symbol. If it is the same symbol, I mark it and return to the left side to repeat the process. Because this variant allows only even-length palindromes, if a single unmarked symbol is left in the middle, the machine rejects.

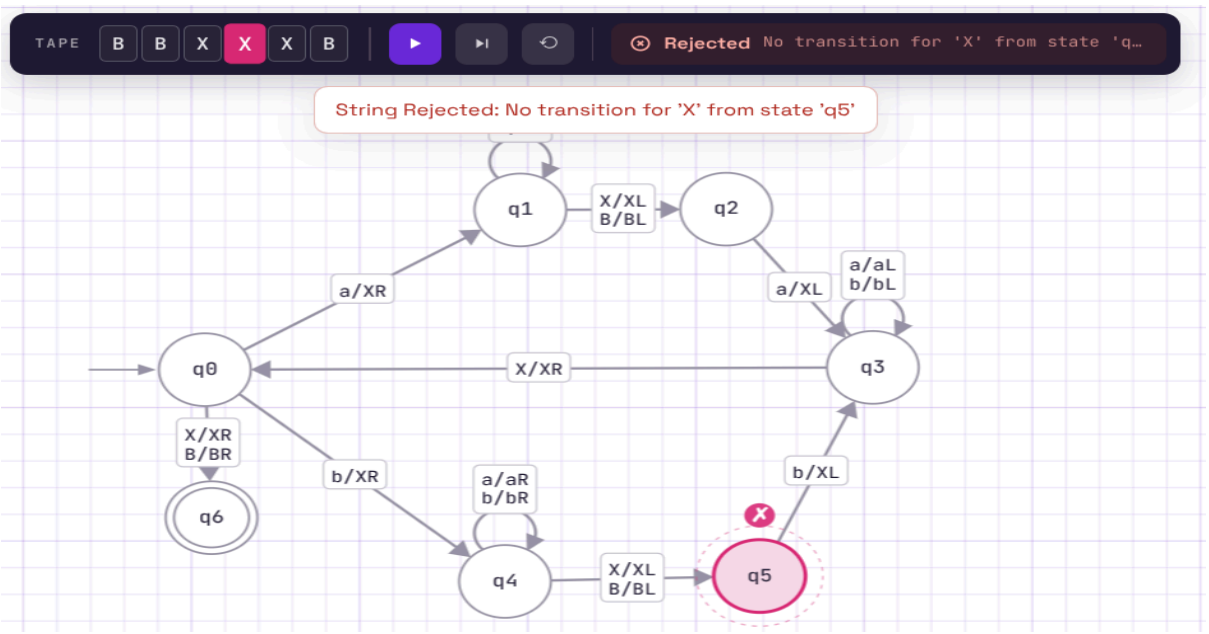
TASK:B

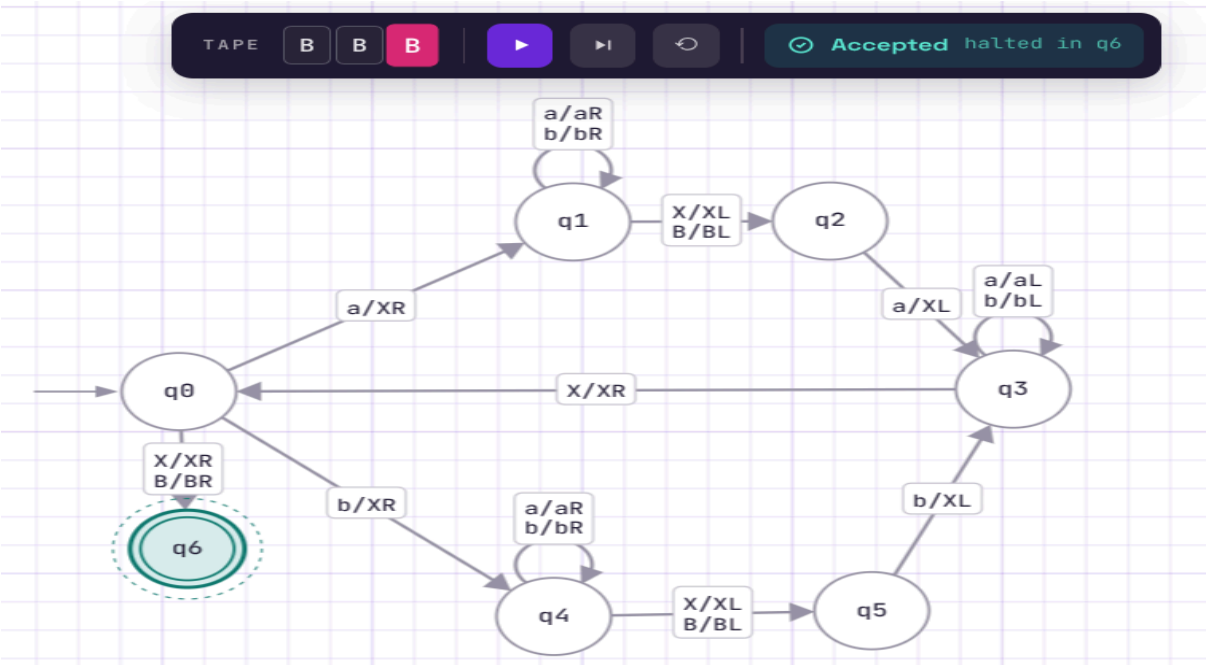


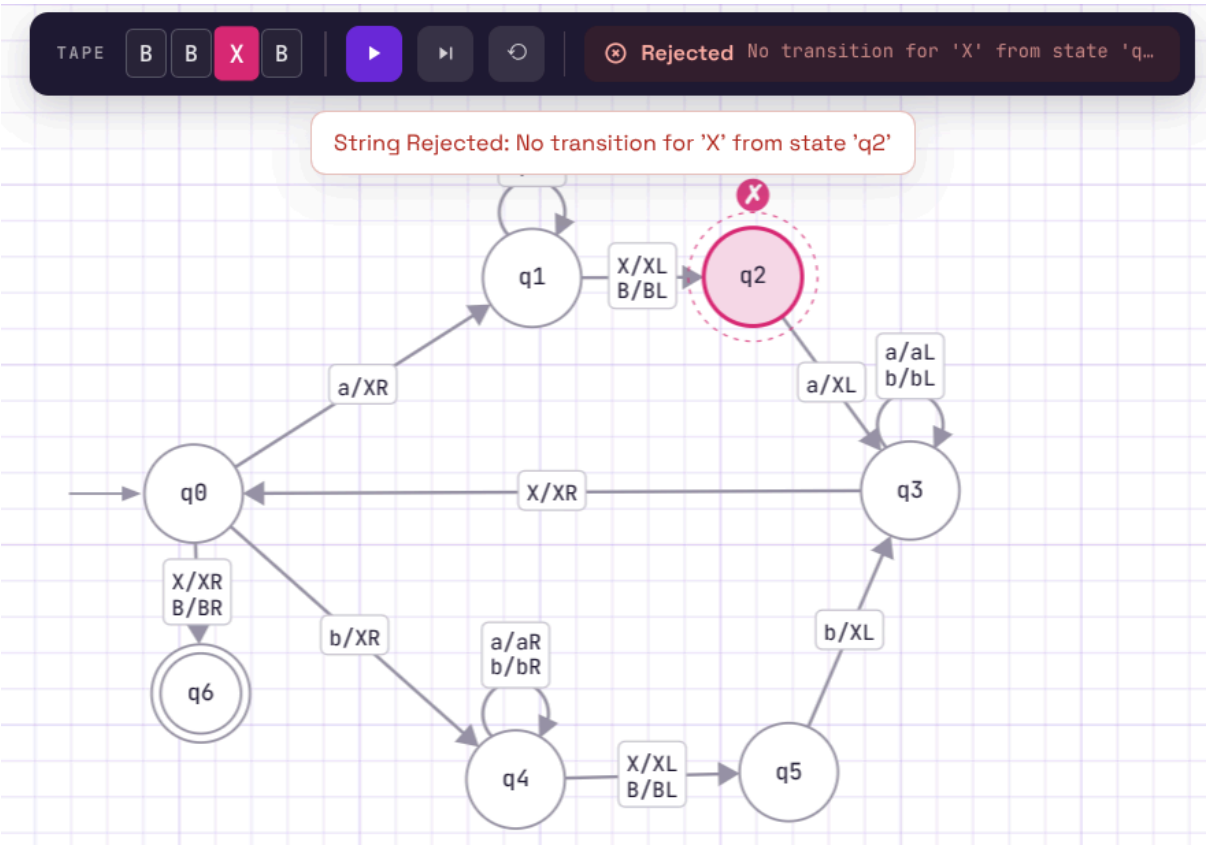
TASK:C




TAPE
B B X X X X X X X X X X X B
Accepted halted in q6
a/aR b/bR
q1
X/XL B/BL
q2
X/XL
a/XR
a/XL
a/aL b/bL
q7
q0
X/XR
q3
X/XR B/BR
X/XL
X/XR B/BR
b/XR
b/XL
a/aR b/bR
q6
q4
X/XL B/BL
q5


TAPE
B B X X X B
Rejected No transition for 'X' from state 'q…
String Rejected: No transition for 'X' from state 'q5'
q1
X/XL B/BL
q2
a/XR
a/XL
a/aL b/bL
q0
X/XR
q3
X/XR B/BR
b/XR
a/aR b/bR
b/XL
q6
q4
X/XL B/BL
q5


TAPE
B B B
Accepted halted in q6
a/aR b/bR
q1
X/XL B/BL
q2
a/XR
a/XL
a/aL b/bL
q0
X/XR
q3
X/XR B/BR
b/XR
a/aR b/bR
b/XL
q6
q4
X/XL B/BL
q5



LINK:
https://www.automataverse.com/m/ByMScFFs

TASK:D

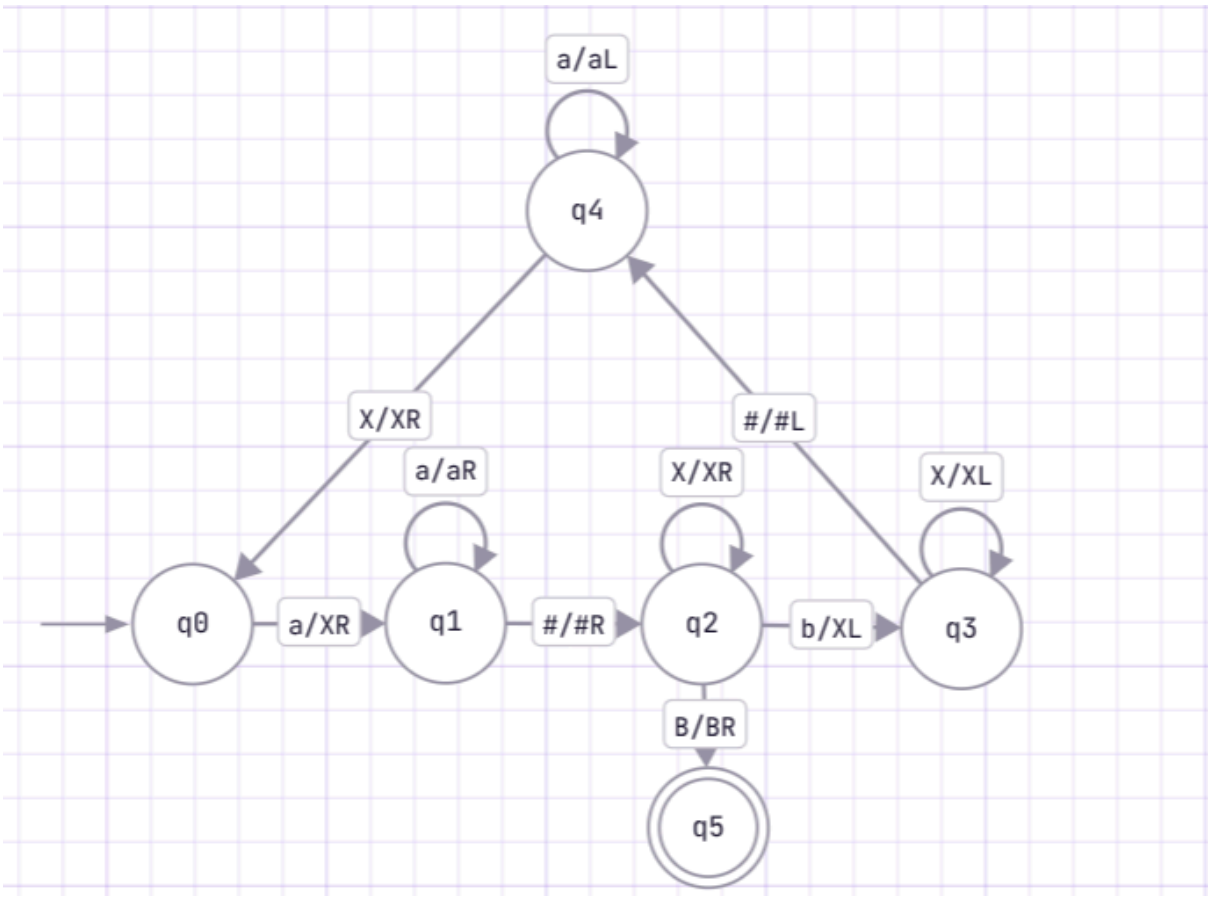
The machine cannot decide a palindrome by looking only from left to right because a symbol near the beginning has to be compared with a symbol near the end. After making a comparison, the machine has to return toward the left and find the next unused pair. Therefore, several left-right sweeps are necessary. With only one left-to-right pass, the machine would not be able to directly compare each symbol with its corresponding symbol at the opposite end.

QUESTION NO: 05

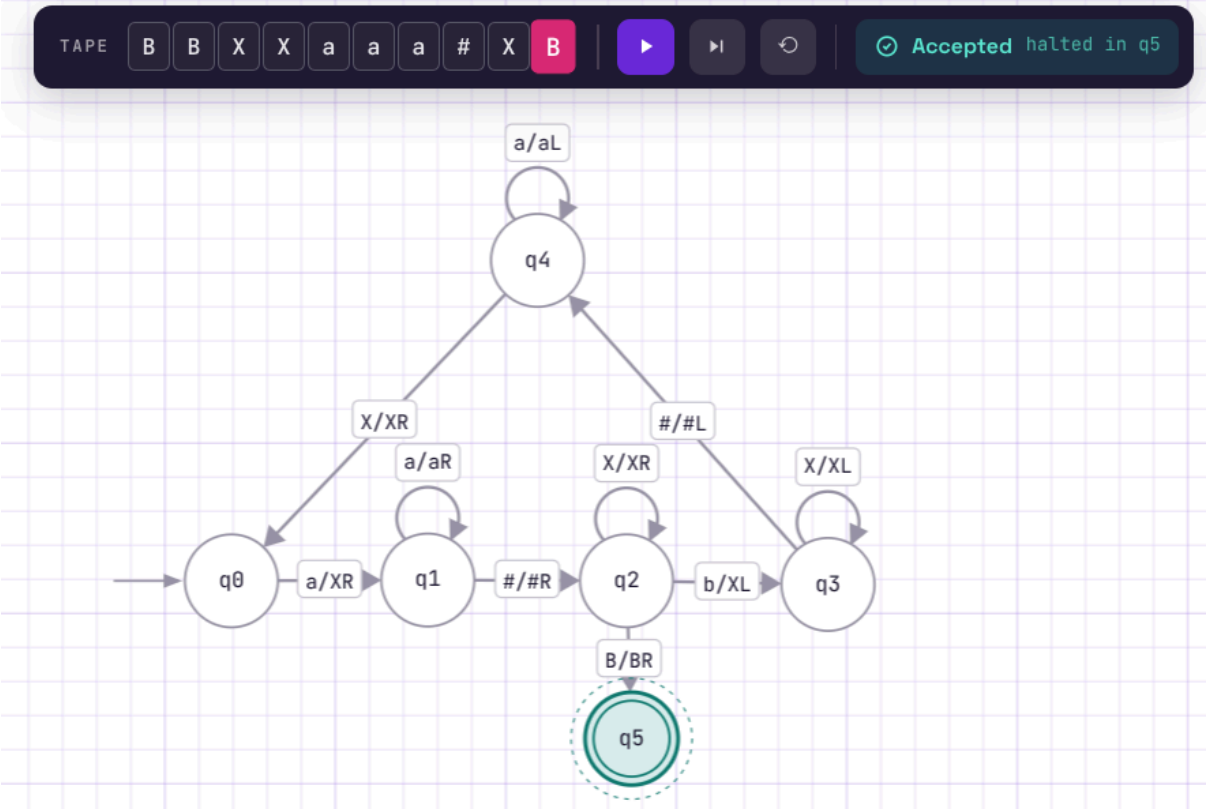
TASK:A

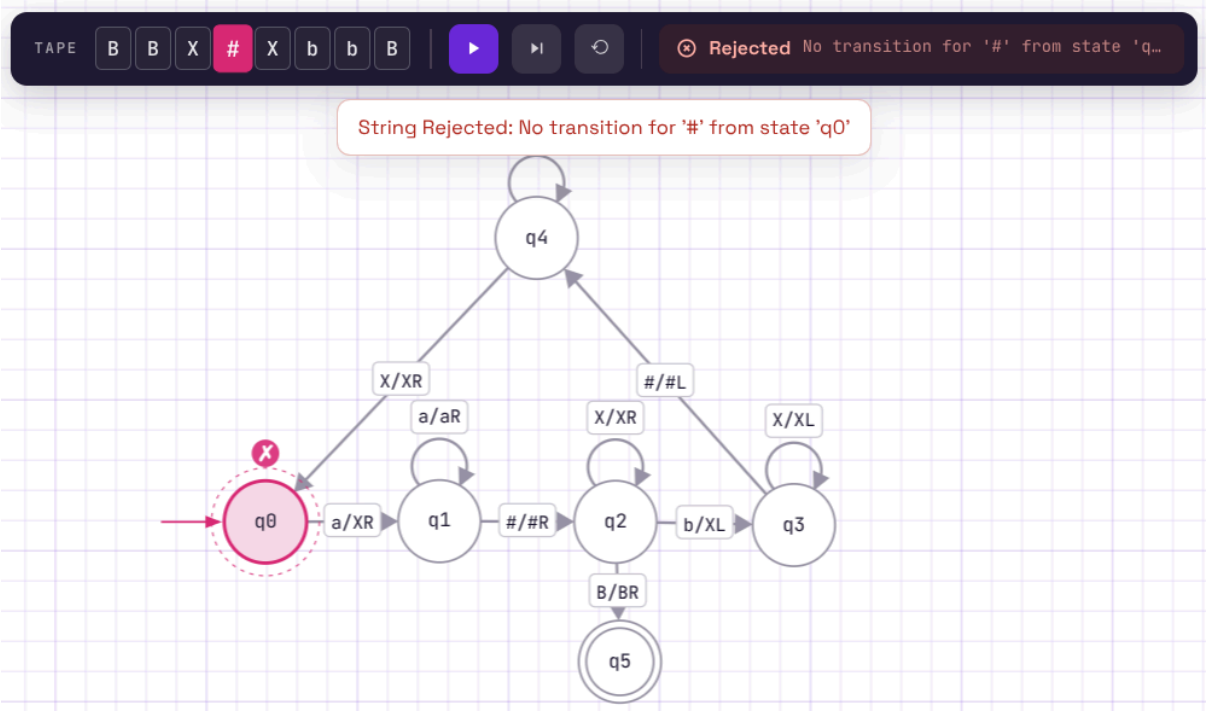
My machine compares the number of a on the left with the number of bs on the right. It repeatedly marks one unused a and then searches for one unused b after the #. I continue this process until there are no more unmarked bs. At that point, the machine checks whether at least one unmarked a is still left. If an extra a remains, then there are more as than bs, so the machine accepts; if both sides finish at the same time or number of b's is more then number of a's, it rejects.

TASK:B



TASK:C





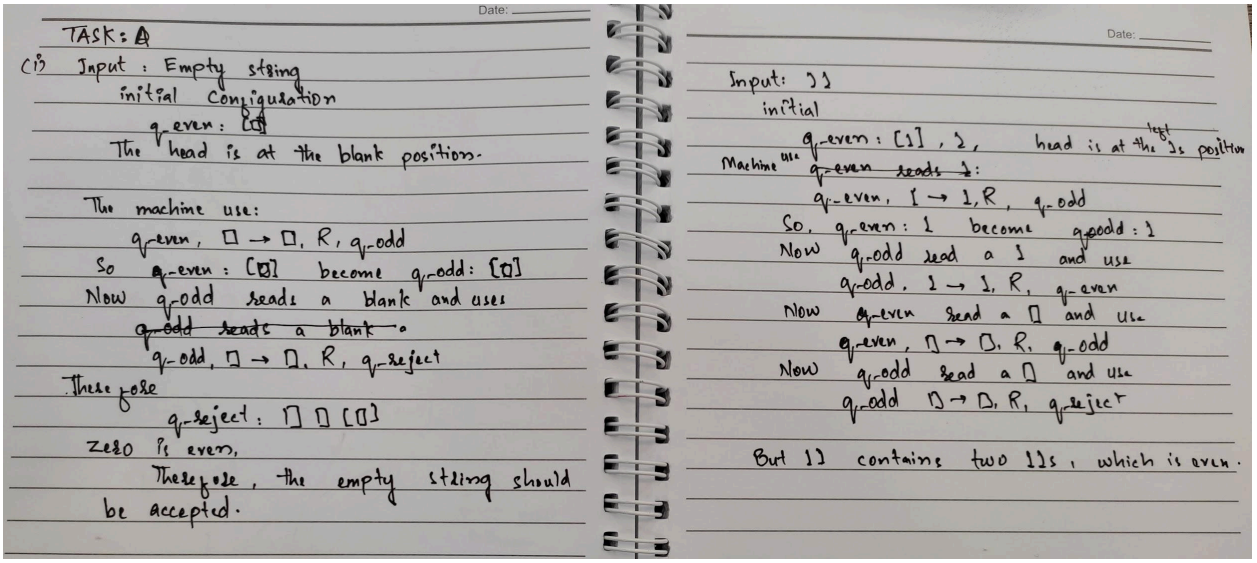
LINK:
https://www.automataverse.com/m/Mg8JZhub

TASK:D

A single left-to-right pass is not enough because the machine has to compare symbols on opposite sides of the #. It needs to remember which as have already been matched with bs, so I mark them and move back and forth across the tape. The important part for this variant is that after all bs are matched, there must still be at least one unused a. That remaining a is what proves that `i > j`.

QUESTION NO: 06

TASK:A



TASK:B

The incorrect row is:

q_even(start)	|_|	|_|	R	q_odd

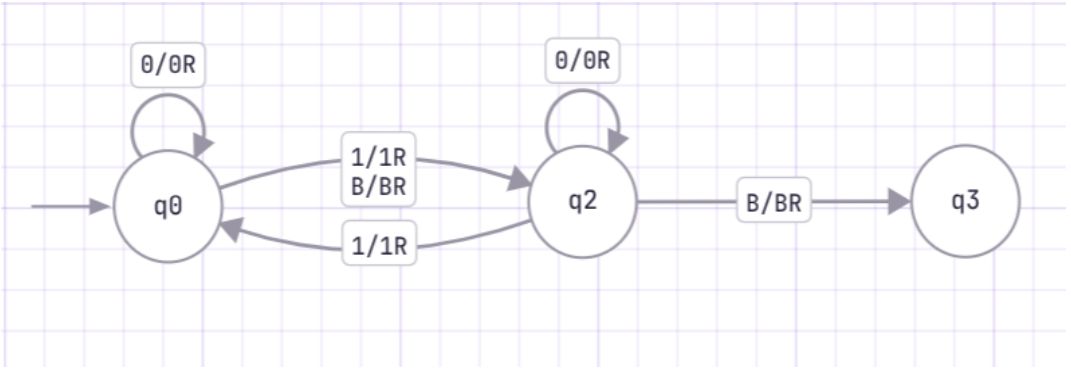
It should instead be:

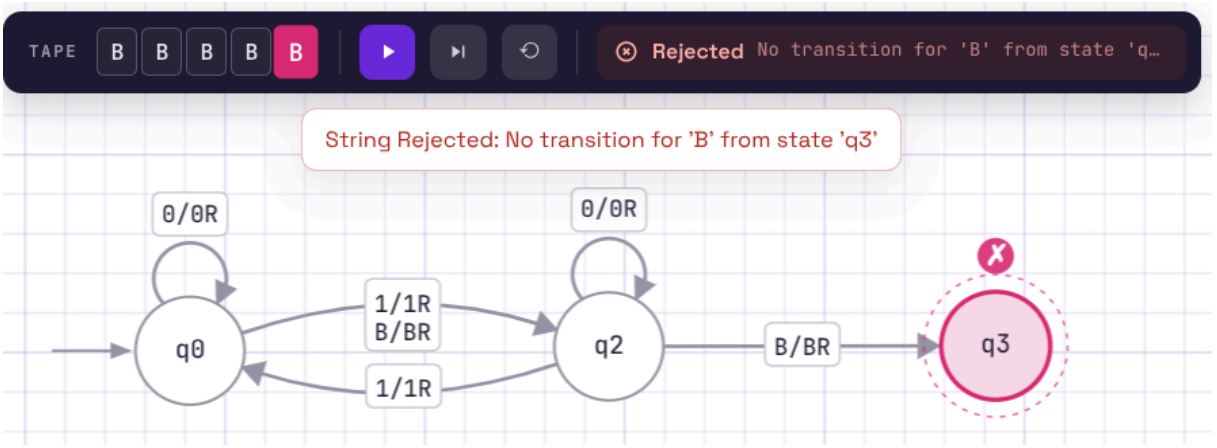
q_even(start)	|_|	|_|	R	q_accept

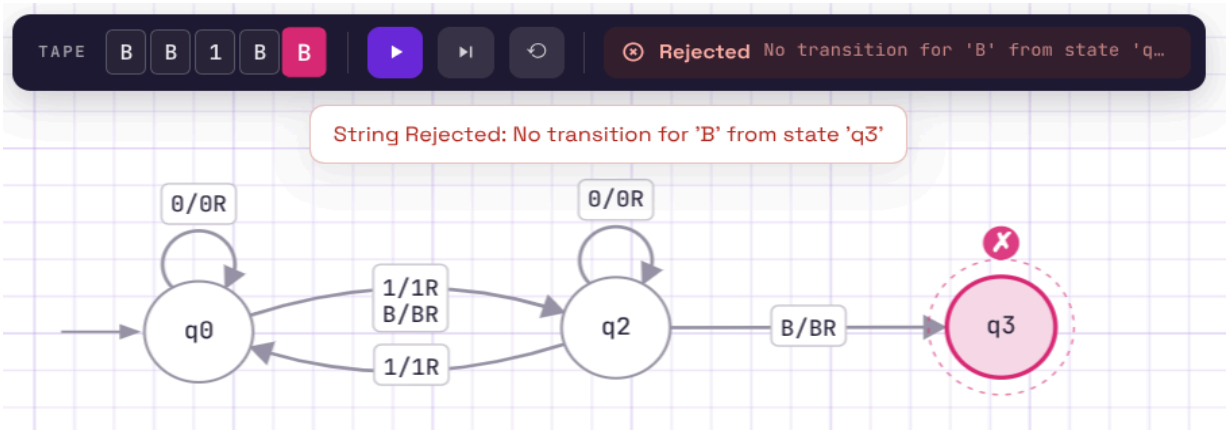
The traces reveal this because whenever the machine reaches the end while it is in q_even, the number of 1s seen so far is even. Therefore, reaching the blank from q_even should mean the input is valid and the machine should accept.

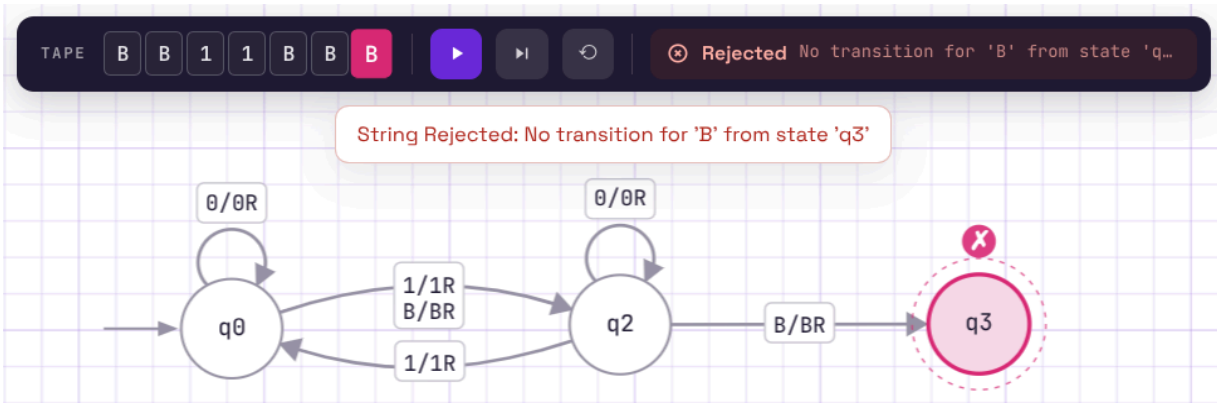
TASK:C

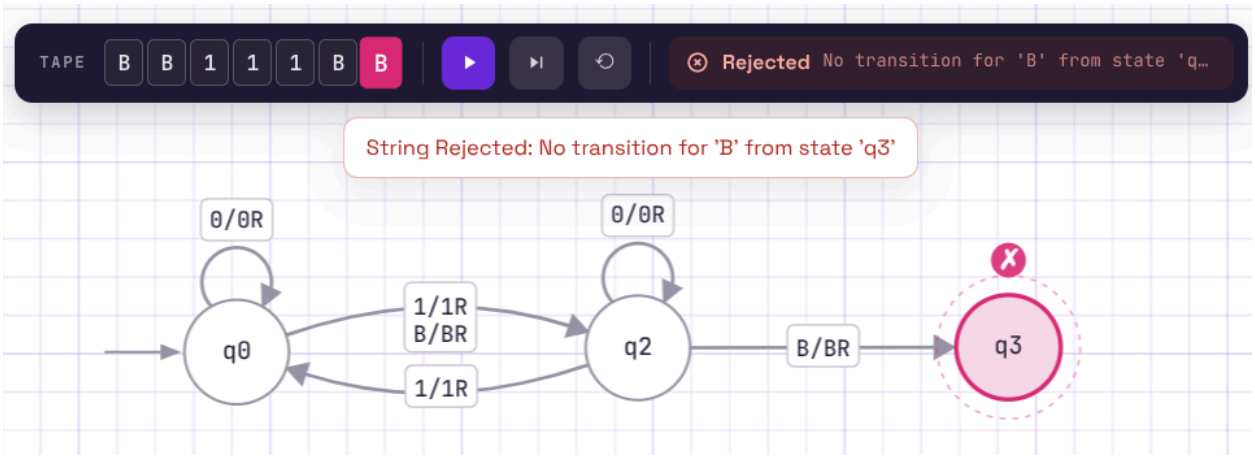
Incorrect Machine





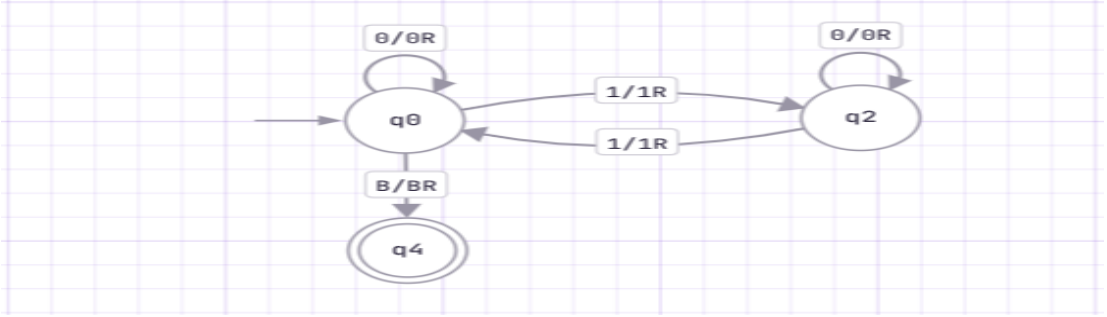


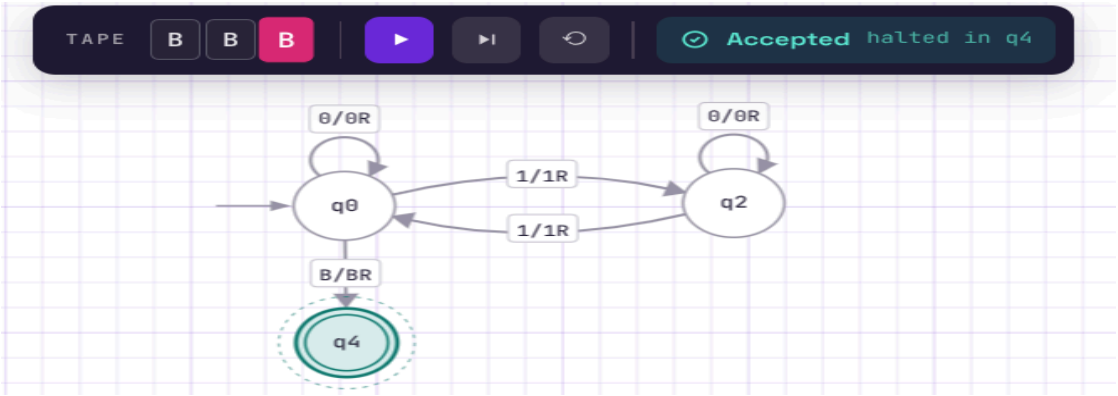


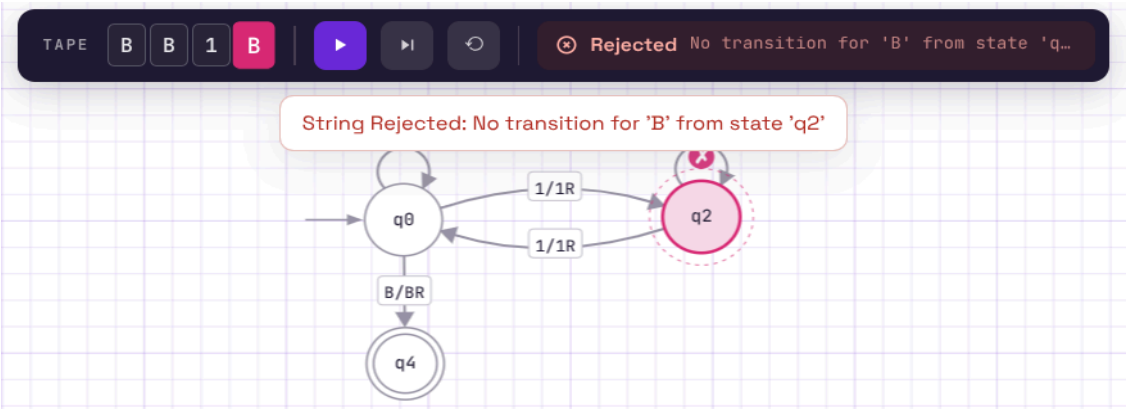


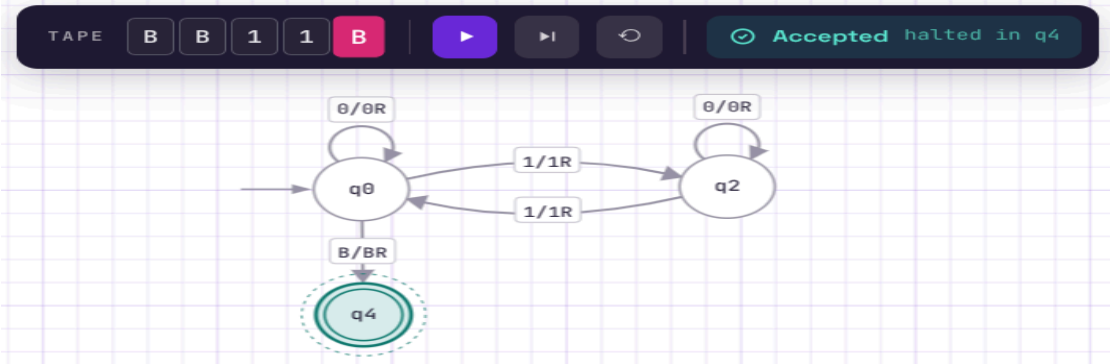
LINK:
https://www.automataverse.com/m/ite48qgt

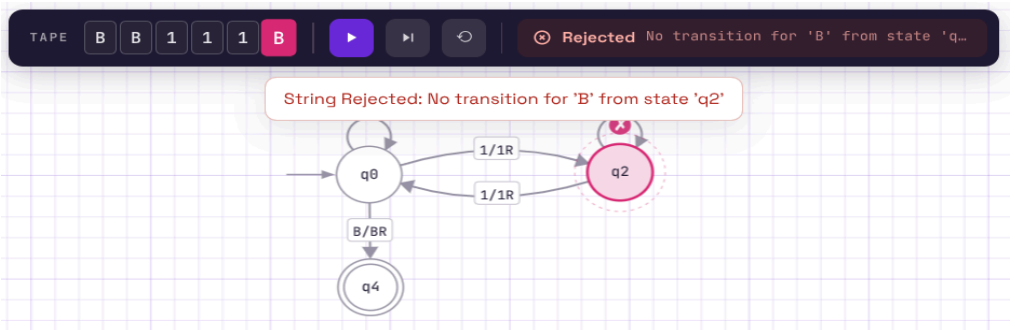
Correct Machine











LINK:
https://www.automataverse.com/m/4o8X9x6n

TASK:D

The inputs 1 and 111 alone would not reveal the bug because both contain an odd number of 1 and should correctly be rejected by both machines. The bug appears when the machine finishes the input while it is in q_even. The empty string and 11 are therefore important test cases because they reach q_even at the blank symbol and expose the incorrect transition. This shows that test cases should include both accepting and rejecting cases, especially boundary cases such as the empty input.